

Verifpal analysis of 02-unbound-statement.vp

August 31, 2026

Abstract

This report describes the analysis of the Verifpal model `02-unbound-statement.vp` against an active attacker, with each principal running 2 concurrent sessions. Of its 2 queries, 2 were broken. Every attack reported here is a witness against the model as written. A query reported as holding means that this search found no attack at these parameters, which is not a proof that none exists.

1 02-unbound-statement.vp

Analysed against an active attacker at 2 sessions per principal, in 113 ms. The result code is `a1c1`.

1.1 Protocol

1.2 Verdicts

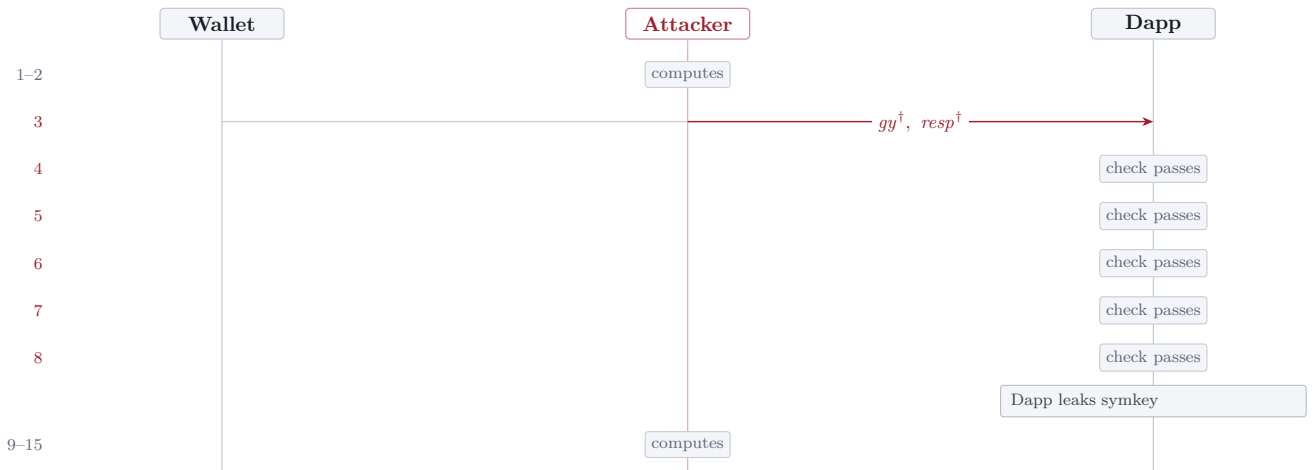
Table 1: Every query in the model, with the verdict this run reached.

Query	Verdict	Scope of the verdict	Line
authentication? Wallet $\rightarrow$ Dapp: <i>resp</i>	attack	a witness, shown below	187
confidentiality? <i>txdata</i>	attack	a witness, shown below	188

1.3 Attacks

1.3.1 authentication? Wallet $\rightarrow$ Dapp: *resp*

resp ($\text{AEAD}_{\text{ENC}}(r_d, de_e, c_ad)$), sent by Attacker and not by Wallet, is successfully used in $\text{AEAD}_{\text{DEC}}^?(r_d, resp, c_ad)$ within Dapp's state.



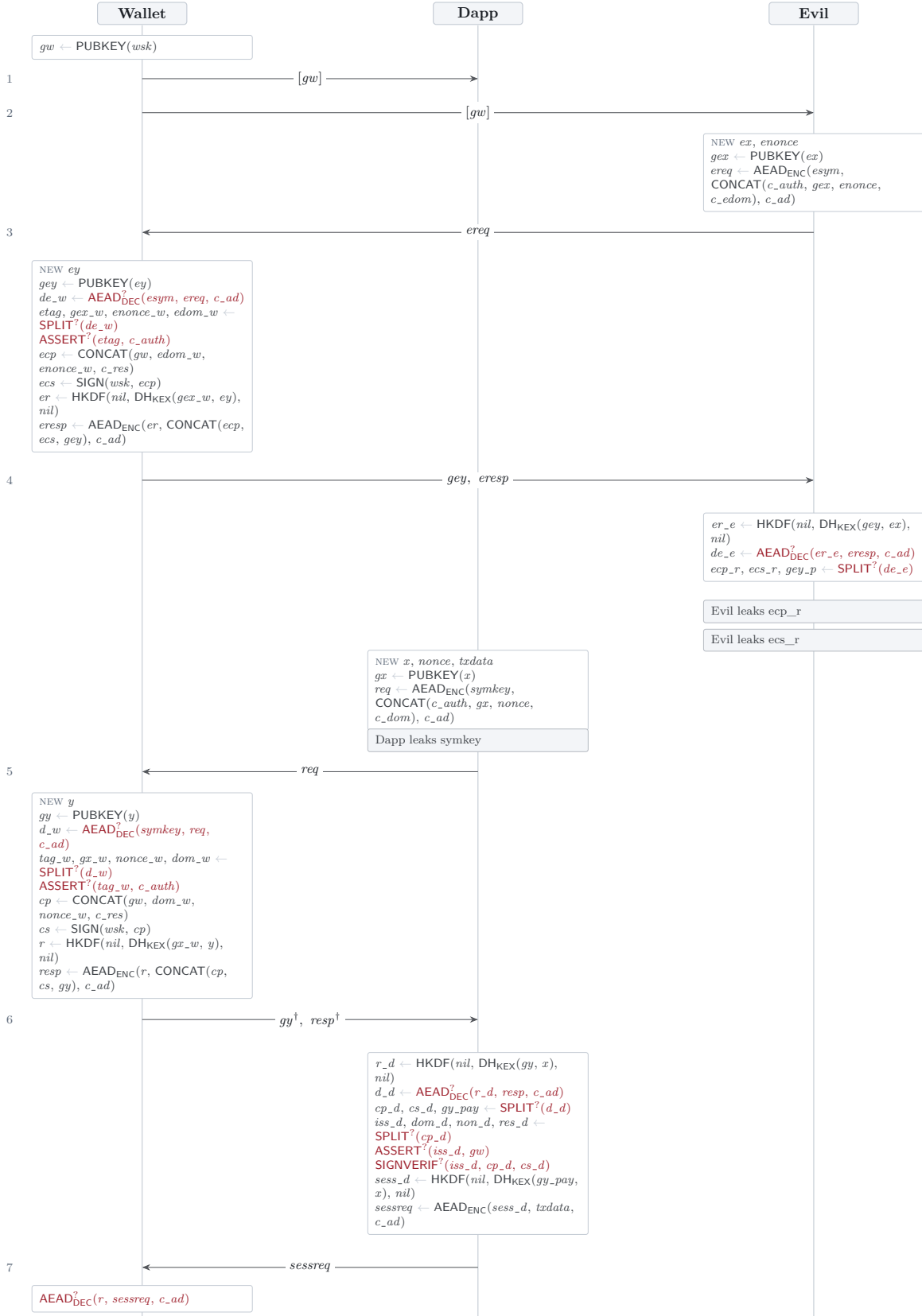


Figure 1: The protocol as written. Guarded values are bracketed; a dagger marks every value an attack below substitutes or replays.

Figure 2: How the attacker breaks authentication? Wallet $\rightarrow$ Dapp: *resp*. Substituted values carry a dagger.

- 1 Attacker constructs $\text{PUBKEY}(\text{nil})$.
- 2 Attacker replaced *gy* with $\text{PUBKEY}(\text{nil})$, after which $\text{AEAD}_{\text{ENC}}(r_d, de_e, c_{ad})$ became available.
- 3 Attacker replaces *gy*, *resp* (sent by Wallet to Dapp) with $\text{PUBKEY}(\text{nil})$, $\text{AEAD}_{\text{ENC}}(r_d, de_e, c_{ad})$. (*gy* was $\text{PUBKEY}(y)$; *resp* was $\text{AEAD}_{\text{ENC}}(r, \text{CONCAT}(cp, cs, gy), c_{ad})$) Wallet $\rightarrow$ Dapp
- 4 Dapp's $\text{AEAD}_{\text{DEC}}^?(r_d, \text{AEAD}_{\text{ENC}}(r_d, de_e, c_{ad}), c_{ad})$ passes — the attacker controls one of its inputs.
- 5 Dapp's $\text{SPLIT}^?(d_d)$ passes — the attacker controls one of its inputs.
- 6 Dapp's $\text{SPLIT}^?(cp_d)$ passes — the attacker controls one of its inputs.
- 7 Dapp's $\text{ASSERT}^?(iss_d, gw)$ passes — the attacker controls one of its inputs.
- 8 Dapp's $\text{SIGNVERIF}^?(iss_d, cp_d, cs_d)$ passes — the attacker controls one of its inputs.
- 9 Attacker observes *req* on the wire.
- 10 Attacker is handed *symkey* by a leaks declaration.
- 11 Attacker opens *req* with *symkey*, obtaining *d_w*.
- 12 Attacker splits *d_w* and takes *gx*.
- 13 Attacker constructs $\text{DH}_{\text{KEX}}(gx, \text{nil})$.
- 14 Attacker constructs *r_d*.
- 15 Attacker opens $\text{AEAD}_{\text{ENC}}(r_d, de_e, c_{ad})$ with *r_d*, obtaining *de_e*.

1.3.2 confidentiality? *txdata*

txdata (*txdata*) is obtained by Attacker.

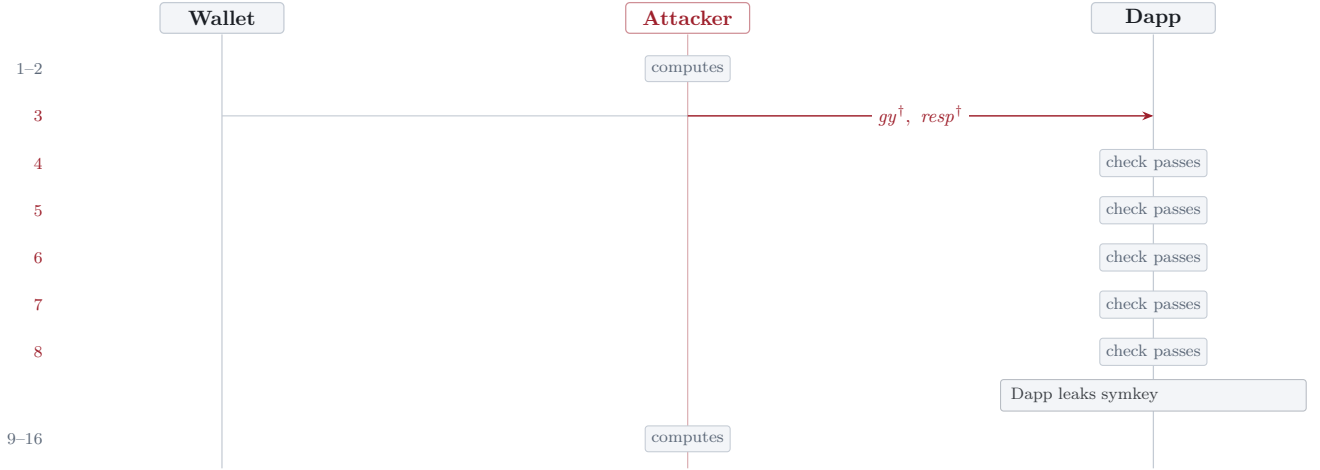


Figure 3: How the attacker breaks confidentiality? *txdata*. Substituted values carry a dagger.

- 1 Attacker constructs $\text{PUBKEY}(\text{nil})$.
- 2 Attacker replaced *gy* with $\text{PUBKEY}(\text{nil})$, after which $\text{AEAD}_{\text{ENC}}(r_d, d_d, c_{ad})$ became available.
- 3 Attacker replaces *gy*, *resp* (sent by Wallet to Dapp) with $\text{PUBKEY}(\text{nil})$, $\text{AEAD}_{\text{ENC}}(r_d, d_d, c_{ad})$. (*gy* was $\text{PUBKEY}(y)$; *resp* was $\text{AEAD}_{\text{ENC}}(r, \text{CONCAT}(cp, cs, gy), c_{ad})$) Wallet $\rightarrow$ Dapp
- 4 Dapp's $\text{AEAD}_{\text{DEC}}^?(r_d, \text{AEAD}_{\text{ENC}}(r_d, \text{CONCAT}(ecp, ecs, \text{PUBKEY}(\text{nil})), c_{ad}), c_{ad})$ passes — the attacker controls one of its inputs.
- 5 Dapp's $\text{SPLIT}^?(d_d)$ passes — the attacker controls one of its inputs.
- 6 Dapp's $\text{SPLIT}^?(cp_d)$ passes — the attacker controls one of its inputs.
- 7 Dapp's $\text{ASSERT}^?(iss_d, gw)$ passes — the attacker controls one of its inputs.
- 8 Dapp's $\text{SIGNVERIF}^?(iss_d, cp_d, cs_d)$ passes — the attacker controls one of its inputs.
- 9 Attacker observes *sessreq* on the wire.
- 10 Attacker observes *req* on the wire.
- 11 Attacker is handed *symkey* by a leaks declaration.

- 12 Attacker opens *req* with *symkey*, obtaining *d_w*.
- 13 Attacker splits *d_w* and takes *gx*.
- 14 Attacker constructs $\text{DH}_{\text{KEX}}(gx, \text{nil})$.
- 15 Attacker constructs *r_d*.
- 16 Attacker opens *sessreq* with *r_d*, obtaining *txdata*.

1.4 Scope and assumptions

Every verdict above was reached against an active attacker, with each principal running 2 concurrent sessions, over exactly the model as written. An attack is a witness and stands on its own. A query reported as holding says only that this search found no attack at those parameters: the search space this engine defines was explored, which is never the space of all attacks.

A Model source

A.1 02-unbound-statement.vp

```

1 // FINDING 2 -a CACAO issued for a DIFFERENT relying party, with a different
2 // domain and a different nonce, is accepted and establishes a session. Fixing
3 // Finding 1's missing `return` does not help, because the check that then runs
4 // is one of internal consistency only.
5 //
6 // Expected: a1c1 at one session and at two.
7 //
8 // -----
9 // WHY THIS SURVIVES THE OBVIOUS FIX
10 // -----
11 //
12 // utils/src/cacao.ts:
13 //
14 // export async function validateSignedCacao({ cacao, projectId }) {
15 //   const { s: signature, p: payload } = cacao;
16 //   reconstructed = formatMessage(payload, payload.iss);
17 //   const walletAddress = getDidAddress(payload.iss);
18 //   return await verifySignature(walletAddress, reconstructed, signature,
19 //     getNamespacedDidChainId(payload.iss), projectId);
20 // }
21 //
22 // The message verified against is rebuilt from the CACAO's OWN payload. The
23 // function answers exactly one question -- "did the address named in `iss`
24 // sign this text?" -- and nothing ties that text to the request that was sent.
25 //
26 // This is not an omission at one call site. Across
27 // sign-client/src/controllers/engine.ts, `nonce` appears only where a request
28 // is constructed (lines 310, 323, 964, 1042) and inside isValidAuthenticate's
29 // input validation (3277, 3289), never in a comparison against a returned
30 // CACAO. `domain` and `aud` are never compared at all. The dapp-side
31 // onAuthenticate handler reads `cacaos` and `responder` out of the result and
32 // goes straight to signature verification, account extraction and session
33 // construction.
34 //
35 // The SDK does hand `auths` back to the caller, so an application COULD run
36 // the SIWE relying-party checks itself. But the session is built, subscribed
37 // and persisted before the promise settles, so an application that performs
38 // those checks afterwards is checking something the SDK has already acted on.
39 //

```

```

40 // This model therefore uses the CHECKED form, SIGNVERIF(...)?, which is what
41 // the code does once Finding 1's missing `return` is added. The attack does
42 // not depend on that bug at all.
43 //
44 // -----
45 // THE ATTACK
46 // -----
47 //
48 // Evil is a relying party the user genuinely signs in to -- a site the
49 // attacker runs. That sign-in is entirely honest: the wallet checks the
50 // request, the user approves, and the wallet issues a CACAO naming Evil's
51 // domain and Evil's nonce. Evil holds it legitimately.
52 //
53 // Evil then presents that same CACAO to an unrelated honest dapp. The dapp
54 // verifies the signature -- it is genuine -- and never notices that the
55 // statement names somebody else's domain and a nonce it never issued. It
56 // installs a session for the user's account, keyed to a public key Evil chose.
57 //
58 // Modelling notes, all as in 01-signature-bypass.vp: HKDF with neither salt
59 // nor info; type 1 envelopes carrying senderPublicKey in the clear outside the
60 // AEAD; the response sealed under generateSharedKey(gy, gx), which is the
61 // session key; the responder public key appearing both in the envelope (which
62 // decides decryption) and inside the ciphertext (which decides the session
63 // topic), never compared.
64 //
65 // PRECONDITION: to deliver the response the attacker needs gx, which rides in
66 // the request under the pairing symKey, hence `leaks symkey` -- the pairing URI
67 // is displayed as a QR code. Over link mode the request is a cleartext type 2
68 // envelope in a URL and this precondition disappears.
69
70 attacker[active]
71
72 principal Wallet[
73     knows private wsk
74     gw = PUBKEY(wsk)
75 ]
76
77 Wallet → Dapp: [gw]
78 Wallet → Evil: [gw]
79
80 // =====
81 // Exchange 1: the user honestly signs in to Evil's site.
82 // =====
83
84 principal Evil[
85     knows private esym // Evil's own pairing with the wallet
86     knows public c_ad
87     knows public c_auth
88     knows public c_edom // Evil's domain
89     knows public c_res
90     generates ex
91     generates enonce // Evil's nonce
92     gex = PUBKEY(ex)
93     ereq = AEAD_ENC(esym, CONCAT(c_auth, gex, enonce, c_edom), c_ad)
94 ]
95
96 Evil → Wallet: ereq
97
98 principal Wallet[
99     knows private esym

```

```

100     knows public c_ad
101     knows public c_auth
102     knows public c_res
103     generates ey
104     gey = PUBKEY(ey)
105     de_w = AEAD_DEC(esym, ereq, c_ad)?
106     etag, gex_w, enonce_w, edom_w = SPLIT(de_w)?
107     _ = ASSERT(etag, c_auth)?
108     ecp = CONCAT(gw, edom_w, enonce_w, c_res)
109     ecs = SIGN(wsk, ecp)
110     er = HKDF(nil, DH_KEX(gex_w, ey), nil)
111     eresp = AEAD_ENC(er, CONCAT(ecp, ecs, gey), c_ad)
112 ]
113
114 Wallet → Evil: gey, eresp
115
116 principal Evil[
117     er_e = HKDF(nil, DH_KEX(gey, ey), nil)
118     de_e = AEAD_DEC(er_e, eresp, c_ad)?
119     ecp_r, ecs_r, gey_p = SPLIT(de_e)?
120     // Evil is the attacker: what it holds legitimately, the attacker holds.
121     leaks ecp_r
122     leaks ecs_r
123 ]
124
125 // =====
126 // Exchange 2: an unrelated honest dapp authenticates the same account.
127 // =====
128
129 principal Dapp[
130     knows private symkey
131     knows public c_ad
132     knows public c_auth
133     knows public c_dom // the honest dapp's domain, distinct from c_edom
134     generates x
135     generates nonce // the honest dapp's nonce, never issued to Evil
136     generates txdata
137     gx = PUBKEY(x)
138     req = AEAD_ENC(symkey, CONCAT(c_auth, gx, nonce, c_dom), c_ad)
139     leaks symkey
140 ]
141
142 Dapp → Wallet: req
143
144 principal Wallet[
145     knows private symkey
146     knows public c_dom
147     generates y
148     gy = PUBKEY(y)
149     d_w = AEAD_DEC(symkey, req, c_ad)?
150     tag_w, gx_w, nonce_w, dom_w = SPLIT(d_w)?
151     _ = ASSERT(tag_w, c_auth)?
152     cp = CONCAT(gw, dom_w, nonce_w, c_res)
153     cs = SIGN(wsk, cp)
154     r = HKDF(nil, DH_KEX(gx_w, y), nil)
155     resp = AEAD_ENC(r, CONCAT(cp, cs, gy), c_ad)
156 ]
157
158 Wallet → Dapp: gy, resp
159

```

```

160 principal Dapp[
161   r_d = HKDF(nil, DH_KEX(gy, x), nil)
162   d_d = AEAD_DEC(r_d, resp, c_ad)?
163   cp_d, cs_d, gy_pay = SPLIT(d_d)?
164   iss_d, dom_d, non_d, res_d = SPLIT(cp_d)?
165   // The dapp is authenticating a PARTICULAR account -- it is re-authenticating
166   // a known user, or matching the address against its own records. Without
167   // this the queries below fail for an uninteresting reason: anyone may
168   // answer an authenticate request with a CACAO for their OWN account, and
169   // will of course hold the resulting session. That is connecting a wallet,
170   // not impersonating one. This assertion is what makes the failure mean
171   // "the dapp accepted a session for gw".
172   _ = ASSERT(iss_d, gw)?
173   // FINDING 1 REPAIRED: this check now halts. It still only asks whether the
174   // address named in the payload signed the payload.
175   _ = SIGNVERIF(iss_d, cp_d, cs_d)?
176   sess_d = HKDF(nil, DH_KEX(gy_pay, x), nil)
177   sessreq = AEAD_ENC(sess_d, txdata, c_ad)
178 ]
179
180 Dapp → Wallet: sessreq
181
182 principal Wallet[
183   _ = AEAD_DEC(r, sessreq, c_ad)?
184 ]
185
186 queries[
187   authentication? Wallet → Dapp: resp
188   confidentiality? txdata
189 ]

```

B On soundness

Verifpal is sound but incomplete. Every attack shown here is a genuine attack on the model as written; a query reported as holding means no attack was found within the search this run performed, which is never a proof that none exists.